Final Project Report: Proximity Detection & Smart Feedback System

Engineer: Stanley Etienne / J00974053

Course: ECEL-360

Date: 04-18-26

Objective

The objective of this project was to design and implement a multi-component embedded system using the ESP32 that detects proximity using an ultrasonic sensor, visually represents distance using an LED bar graph, and communicates system status through wireless and display interfaces.

The system integrates multiple concepts from previous labs, including sensor input, GPIO control, wireless communication, and real-time data processing, to create a complete smart detection system.

System Architecture

The system consists of the following components:

- Ultrasonic Sensor: Measures distance to objects
- LED Bar Graph: Displays proximity visually
- ESP32: Processes data and controls outputs
- Communication Interfaces: Wi-Fi/BLE/LCD for feedback

Flow:
1. Sensor measures distance
2. ESP32 processes data
3. LED bar graph updates
4. System triggers an alert when the threshold is reached

Concepts Implemented

- Ultrasonic sensing (distance measurement)
- LED bar graph mapping (data visualization)
- Wireless communication (Wi-Fi/BLE)

- LCD display (user interface)
- Real-time processing (continuous updates)

System Design Explanation

The ESP32 continuously reads the distance from the ultrasonic sensor and maps the value to a range of 0–10 LEDs. As the object gets closer, more LEDs turn on.

When all LEDs are on, the system recognizes that an object is too close and triggers a response such as sending a signal or displaying a message.

Results

The system operated successfully during testing. The ultrasonic sensor provided stable readings, and the LED bar graph accurately displayed proximity.

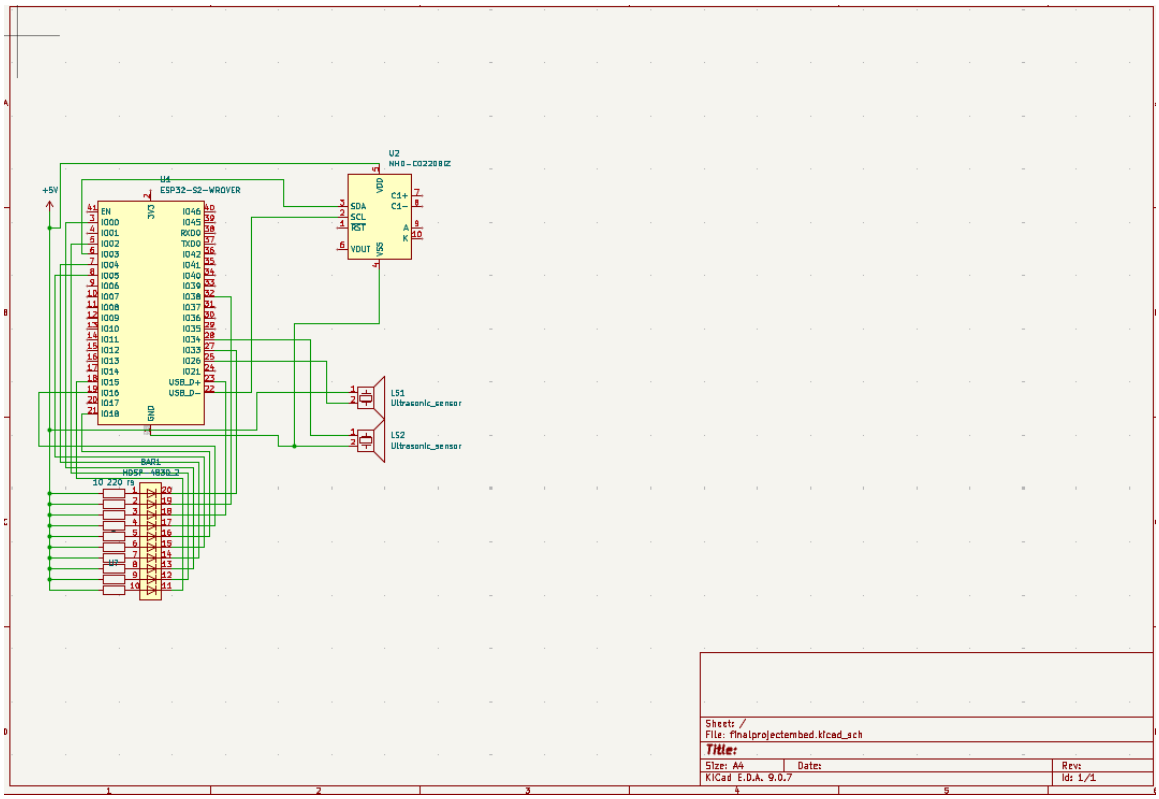
The system responded immediately to changes in distance and successfully detected when an object reached the threshold.

Conclusion

This project successfully combined multiple embedded systems concepts into one working design. The ESP32 acted as a central controller integrating sensing, processing, and output.

The project demonstrated real-world applications such as smart monitoring and proximity detection systems.

Schematic and Picture:



Code:

```
/*
  Final Project: Proximity Detection System
  Engineer: Stanley Etienne

  Description:
  - Ultrasonic sensor measures distance
  - LED bar graph displays proximity
  - Triggers alert when object is too close
*/

#include <WiFi.h>
#include <WebServer.h>
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

#include <BLEDevice.h>
#include <BLEServer.h>
#include <BLEUtils.h>
#include <BLE2902.h>
```

```
// ---------------- Pins ----------------
#define TRIG_PIN 27
#define ECHO_PIN 26
#define BUZZER_PIN 25

const int NUM_LEDS = 10;
int ledPins[NUM_LEDS] = {15, 4, 0, 2, 5, 18, 19, 23, 32, 33};

// ---------------- Wi-Fi ----------------
const char* ssid = "WalthallB608";
const char* password = "ewhomeb608!";

// ---------------- Objects ----------------
WebServer server(80);
LiquidCrystal_I2C lcd(0x27, 16, 2);

// ---------------- BLE ----------------
#define DEVICE_NAME            "SE_Security_System"
#define SERVICE_UUID           "12345678-1234-1234-1234-1234567890ab"
#define CHARACTERISTIC_UUID_RX "abcd1234-5678-90ab-cdef-1234567890ab"
#define CHARACTERISTIC_UUID_TX "98765432-4321-4321-4321-ba0987654321"

BLECharacteristic* pTxCharacteristic = nullptr;
bool bleDeviceConnected = false;

// ---------------- Sensor / Display State ----------------
#define MIN_DIST 5
#define MAX_DIST 80
#define NUM_SAMPLES 10

float currentDistance = 0.0;
int currentLedsOn = 0;
String motionStatus = "No Motion";

// Smooth LED fill
float filteredDistance = MAX_DIST;
int displayedLedsOn = 0;
unsigned long lastLedStepTime = 0;
const unsigned long ledStepDelay = 50;
const float distanceAlpha = 0.22;

// ---------------- System State ----------------
bool systemArmed = false;
String alarmMode = "silent";   // "silent" or "regular"
```

```
bool alertActive = false;
bool fullBarLatched = false;
int triggerLimit = 3;

// Separate counters
int disarmedFullHits = 0;
int armedFullHits = 0;

// STATUS screen control
bool showStatusScreen = false;
unsigned long statusScreenStart = 0;
const unsigned long statusScreenDuration = 3000;

// LCD alert rotation
unsigned long lastAlertMsgTime = 0;
int alertMsgIndex = 0;
const unsigned long alertMsgInterval = 1600;

// Buzzer
bool buzzerState = false;
unsigned long lastBuzzerToggle = 0;
const unsigned long buzzerInterval = 250;

// ---------------- Forward Declarations ----------------
float getDistanceCM();
float getSmoothDistance();
void updateBarGraph(int ledsOn);
void updateLCD();
void handleAlarmSound();
void evaluateTriggerLogic();
void sendBleMessage(String msg);
String getStatusReport();

void handleRoot();
void handleData();
void handleArm();
void handleDisarm();
void handleReset();
void handleSetLimit();
void handleSetMode();

// ---------------- Ultrasonic ----------------
float getDistanceCM() {
  digitalWrite(TRIG_PIN, LOW);
  delayMicroseconds(3);
```

```
  digitalWrite(TRIG_PIN, HIGH);
  delayMicroseconds(10);
  digitalWrite(TRIG_PIN, LOW);

  long duration = pulseIn(ECHO_PIN, HIGH, 30000);

  if (duration == 0) {
    return -1;
  }

  return duration * 0.0343 / 2.0;
}

float getSmoothDistance() {
  float readings[NUM_SAMPLES];
  int count = 0;

  for (int i = 0; i < NUM_SAMPLES; i++) {
    float d = getDistanceCM();

    if (d > 0 && d <= 200) {
      readings[count] = d;
      count++;
    }

    delay(5);
  }

  if (count == 0) {
    return -1;
  }

  for (int i = 0; i < count - 1; i++) {
    for (int j = i + 1; j < count; j++) {
      if (readings[j] < readings[i]) {
        float temp = readings[i];
        readings[i] = readings[j];
        readings[j] = temp;
      }
    }
  }

  int start = count / 4;
  int end = (count * 3) / 4;
```

```
  if (end <= start) {
    start = 0;
    end = count;
  }

  float sum = 0;
  int used = 0;

  for (int i = start; i < end; i++) {
    sum += readings[i];
    used++;
  }

  if (used == 0) {
    return -1;
  }

  return sum / used;
}

// ---------------- LED Bar Graph ----------------
void updateBarGraph(int ledsOn) {
  for (int i = 0; i < NUM_LEDS; i++) {
    // Active-low LED bar
    digitalWrite(ledPins[i], (i < ledsOn) ? LOW : HIGH);
  }
}

// ---------------- Trigger Logic ----------------
void evaluateTriggerLogic() {
  if (currentLedsOn == NUM_LEDS && !fullBarLatched) {
    fullBarLatched = true;

    if (systemArmed) {
      armedFullHits++;

      if (armedFullHits >= triggerLimit) {
        alertActive = true;
      }
    } else {
      disarmedFullHits++;
    }
  }
```

```
  if (currentLedsOn < NUM_LEDS) {
    fullBarLatched = false;
  }
}

// ---------------- Alarm Sound ----------------
void handleAlarmSound() {
  if (!alertActive || alarmMode != "regular") {
    noTone(BUZZER_PIN);
    buzzerState = false;
    return;
  }

  unsigned long now = millis();
  if (now - lastBuzzerToggle >= buzzerInterval) {
    lastBuzzerToggle = now;
    buzzerState = !buzzerState;

    if (buzzerState) {
      tone(BUZZER_PIN, 2200);
    } else {
      noTone(BUZZER_PIN);
    }
  }
}

// ---------------- LCD ----------------
void updateLCD() {
  static String lastLine1 = "";
  static String lastLine2 = "";

  String line1 = "";
  String line2 = "";

  if (showStatusScreen) {
    if (millis() - statusScreenStart < statusScreenDuration) {
      if (!systemArmed) {
        line1 = "Disarmed Hits:";
        line2 = String(disarmedFullHits) + " times";
      } else {
        if (armedFullHits >= triggerLimit) {
          line1 = "WE HAD A";
          line2 = "BREACH!!!";
        } else {
          line1 = "Armed Hits:";
```

```
        line2 = String(armedFullHits) + " of " + String(triggerLimit);
      }
    }
  } else {
    showStatusScreen = false;
  }
}

if (!showStatusScreen) {
  if (alertActive) {
    const char* msg1[] = {
      "Ooo you in",
      "You gon get",
      "They on they"
    };

    const char* msg2[] = {
      "trouble",
      "it now",
      "way"
    };

    unsigned long now = millis();
    if (now - lastAlertMsgTime >= alertMsgInterval) {
      lastAlertMsgTime = now;
      alertMsgIndex++;
      if (alertMsgIndex >= 3) alertMsgIndex = 0;
    }

    line1 = msg1[alertMsgIndex];
    line2 = msg2[alertMsgIndex];
  }
  else if (currentLedsOn >= 8 && currentLedsOn <= 9) {
    line1 = "Caught in 4K";
    line2 = "buddy";
  }
  else if (currentLedsOn >= 5 && currentLedsOn <= 6) {
    line1 = "Nah who";
    line2 = "is that?";
  }
  else {
    if (systemArmed) {
      line1 = "ARMED " + String(currentLedsOn) + "/10";
    } else {
      line1 = "DISARM " + String(currentLedsOn) + "/10";
```

```
      }

      line2 = String(currentDistance, 1) + "cm L:" + String(triggerLimit);
    }
  }

  if (line1 != lastLine1 || line2 != lastLine2) {
    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print(line1);
    lcd.setCursor(0, 1);
    lcd.print(line2);

    lastLine1 = line1;
    lastLine2 = line2;
  }
}

// ---------------- BLE Status String ----------------
String getStatusReport() {
  String report = "";
  report += (systemArmed ? "ARMED" : "DISARMED");
  report += " | Last: " + String(currentDistance, 1) + " cm";
  report += " | Bars: " + String(currentLedsOn) + "/10";
  report += " | Armed Hits: " + String(armedFullHits);
  report += " | Disarmed Hits: " + String(disarmedFullHits);
  report += " | Limit: " + String(triggerLimit);
  report += " | Mode: " + alarmMode;
  report += " | Alert: " + String(alertActive ? "YES" : "NO");
  return report;
}

void sendBleMessage(String msg) {
  if (pTxCharacteristic && bleDeviceConnected) {
    pTxCharacteristic->setValue(msg.c_str());
    pTxCharacteristic->notify();
  }
}

// ---------------- BLE Callbacks ----------------
class MyServerCallbacks : public BLEServerCallbacks {
  void onConnect(BLEServer* pServer) override {
    bleDeviceConnected = true;
    Serial.println("BLE connected");
  }
```

```
  void onDisconnect(BLEServer* pServer) override {
    bleDeviceConnected = false;
    Serial.println("BLE disconnected");
    BLEDevice::startAdvertising();
    Serial.println("BLE advertising restarted");
  }
};

class MyCallbacks : public BLECharacteristicCallbacks {
  void onWrite(BLECharacteristic* pCharacteristic) override {
    String cmd = pCharacteristic->getValue().c_str();

    if (cmd.length() == 0) return;

    cmd.trim();
    cmd.toUpperCase();

    Serial.print("BLE Command: ");
    Serial.println(cmd);

    if (cmd == "ARM") {
      systemArmed = true;
      sendBleMessage("System armed.");
    }
    else if (cmd == "DISARM") {
      systemArmed = false;
      alertActive = false;
      noTone(BUZZER_PIN);
      sendBleMessage("System disarmed.");
    }
    else if (cmd == "STATUS") {
      showStatusScreen = true;
      statusScreenStart = millis();

      if (!systemArmed) {
        sendBleMessage("Showing disarmed count on LCD.");
      } else {
        if (armedFullHits >= triggerLimit) {
          sendBleMessage("Showing breach message on LCD.");
        } else {
          sendBleMessage("Showing armed count on LCD.");
        }
      }
    }
```

```
    else if (cmd == "RESET") {
      armedFullHits = 0;
      disarmedFullHits = 0;
      alertActive = false;
      fullBarLatched = false;
      noTone(BUZZER_PIN);
      sendBleMessage("Counters reset.");
    }
    else if (cmd == "SILENT") {
      alarmMode = "silent";
      sendBleMessage("Alarm mode set to SILENT.");
    }
    else if (cmd == "REGULAR") {
      alarmMode = "regular";
      sendBleMessage("Alarm mode set to REGULAR.");
    }
    else {
      sendBleMessage("Commands: ARM DISARM STATUS RESET SILENT REGULAR");
    }
  }
};

void setupBLE() {
  BLEDevice::init(DEVICE_NAME);

  BLEServer* pServer = BLEDevice::createServer();
  pServer->setCallbacks(new MyServerCallbacks());

  BLEService* pService = pServer->createService(SERVICE_UUID);

  pTxCharacteristic = pService->createCharacteristic(
    CHARACTERISTIC_UUID_TX,
    BLECharacteristic::PROPERTY_READ |
    BLECharacteristic::PROPERTY_NOTIFY
  );
  pTxCharacteristic->addDescriptor(new BLE2902());
  pTxCharacteristic->setValue("ESP32 Security Ready");

  BLECharacteristic* pRxCharacteristic = pService->createCharacteristic(
    CHARACTERISTIC_UUID_RX,
    BLECharacteristic::PROPERTY_WRITE
  );
  pRxCharacteristic->setCallbacks(new MyCallbacks());

  pService->start();
```

```
  BLEDevice::startAdvertising();

  Serial.println("BLE ready");
  Serial.println("Device name: SE_Security_System");
}

// ---------------- Web Server ----------------
void handleRoot() {
  String html = R"rawliteral(
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <title>ESP32 Security Monitor</title>
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <style>
    body {
      font-family: Arial, sans-serif;
      text-align: center;
      background: #111;
      color: white;
      margin-top: 40px;
    }

    .card {
      width: 400px;
      max-width: 92%;
      margin: auto;
      padding: 25px;
      border-radius: 16px;
      background: #1e1e1e;
      box-shadow: 0 0 15px rgba(255,255,255,0.1);
    }

    h1 {
      margin-bottom: 20px;
    }

    .status {
      font-size: 28px;
      font-weight: bold;
      margin: 20px 0;
    }

    .motion {
```

```
    color: red;
  }

  .nomotion {
    color: limegreen;
  }

  .armed {
    color: orange;
    font-weight: bold;
  }

  .disarmed {
    color: deepskyblue;
    font-weight: bold;
  }

  .alert-on {
    color: red;
    font-weight: bold;
  }

  .alert-off {
    color: limegreen;
    font-weight: bold;
  }

  .reading {
    font-size: 20px;
    margin: 10px 0;
  }

  .controls {
    margin-top: 20px;
    text-align: left;
  }

  .controls label {
    display: block;
    margin-top: 15px;
    margin-bottom: 5px;
    font-size: 16px;
  }

  input, select, button {
```

```
      width: 100%;
      padding: 10px;
      margin-top: 5px;
      border: none;
      border-radius: 8px;
      font-size: 16px;
      box-sizing: border-box;
    }

    button {
      cursor: pointer;
      background: #333;
      color: white;
      margin-top: 10px;
    }

    button:hover {
      background: #555;
    }

    .button-row {
      display: flex;
      gap: 10px;
      margin-top: 10px;
    }

    .button-row button {
      flex: 1;
    }
  </style>
</head>
<body>
  <div class="card">
    <h1>ESP32 Security Monitor</h1>

    <div id="status" class="status">Loading...</div>
    <div id="distance" class="reading">Distance: -- cm</div>
    <div id="leds" class="reading">LEDs ON: -- / 10</div>
    <div id="armState" class="reading">System: --</div>
    <div id="alarmMode" class="reading">Alarm Mode: --</div>
    <div id="armedHits" class="reading">Armed Hits: --</div>
    <div id="disarmedHits" class="reading">Disarmed Hits: --</div>
    <div id="triggerLimit" class="reading">Trigger Limit: --</div>
    <div id="alertState" class="reading">Alert: --</div>
```

```
    <div class="controls">
      <label for="limitInput">Set Trigger Limit</label>
      <input type="number" id="limitInput" min="1" max="20" value="3">

      <button onclick="setLimit()">Set Limit</button>

      <label for="modeSelect">Alarm Mode</label>
      <select id="modeSelect" onchange="setMode()">
        <option value="silent">Silent Alarm</option>
        <option value="regular">Regular Alarm</option>
      </select>

      <div class="button-row">
        <button onclick="sendCommand('arm')">Arm</button>
        <button onclick="sendCommand('disarm')">Disarm</button>
      </div>

      <div class="button-row">
        <button onclick="sendCommand('reset')">Reset Alert</button>
        <button onclick="updateData()">Refresh</button>
      </div>
    </div>
  </div>

  <script>
    function updateData() {
      fetch('/data')
        .then(response => response.json())
        .then(data => {
          const statusEl = document.getElementById('status');
          statusEl.textContent = data.status;

          if (data.status === "Motion Detected" || data.alertActive ===
true) {
            statusEl.className = "status motion";
          } else {
            statusEl.className = "status nomotion";
          }

          document.getElementById('distance').textContent =
            "Distance: " + data.distance + " cm";

          document.getElementById('leds').textContent =
            "LEDs ON: " + data.ledsOn + " / 10";
```

```
          const armStateEl = document.getElementById('armState');
          armStateEl.textContent = "System: " + (data.armed ? "ARMED" :
"DISARMED");
          armStateEl.className = "reading " + (data.armed ? "armed" :
"disarmed");

          document.getElementById('alarmMode').textContent =
            "Alarm Mode: " + data.alarmMode;

          document.getElementById('armedHits').textContent =
            "Armed Hits: " + data.armedHits;

          document.getElementById('disarmedHits').textContent =
            "Disarmed Hits: " + data.disarmedHits;

          document.getElementById('triggerLimit').textContent =
            "Trigger Limit: " + data.triggerLimit;

          const alertStateEl = document.getElementById('alertState');
          alertStateEl.textContent = "Alert: " + (data.alertActive ?
"ACTIVE" : "CLEAR");
          alertStateEl.className = "reading " + (data.alertActive ?
"alert-on" : "alert-off");

          document.getElementById('limitInput').value = data.triggerLimit;
          document.getElementById('modeSelect').value = data.alarmMode;
        })
        .catch(error => {
          console.log("Error:", error);
        });
    }

    function sendCommand(cmd) {
      fetch('/' + cmd)
        .then(response => response.text())
        .then(result => {
          console.log(result);
          updateData();
        })
        .catch(error => {
          console.log("Error:", error);
        });
    }

    function setLimit() {
```

```
      const value = document.getElementById('limitInput').value;
      fetch('/setLimit?value=' + value)
        .then(response => response.text())
        .then(result => {
          console.log(result);
          updateData();
        })
        .catch(error => {
          console.log("Error:", error);
        });
    }

    function setMode() {
      const mode = document.getElementById('modeSelect').value;
      fetch('/setMode?value=' + mode)
        .then(response => response.text())
        .then(result => {
          console.log(result);
          updateData();
        })
        .catch(error => {
          console.log("Error:", error);
        });
    }

    updateData();
    setInterval(updateData, 1000);
  </script>
</body>
</html>
)rawliteral";

  server.send(200, "text/html", html);
}

void handleData() {
  String statusText = motionStatus;
  if (alertActive) {
    statusText = "ALERT ACTIVE";
  }

  String json = "{";
  json += "\"status\":\"" + statusText + "\",";
  json += "\"distance\":" + String(currentDistance, 2) + ",";
  json += "\"ledsOn\":" + String(currentLedsOn) + ",";
```

```
  json += "\"armed\":" + String(systemArmed ? "true" : "false") + ",";
  json += "\"alarmMode\":\"" + alarmMode + "\",";
  json += "\"armedHits\":" + String(armedFullHits) + ",";
  json += "\"disarmedHits\":" + String(disarmedFullHits) + ",";
  json += "\"triggerLimit\":" + String(triggerLimit) + ",";
  json += "\"alertActive\":" + String(alertActive ? "true" : "false");
  json += "}";

  server.send(200, "application/json", json);
}

void handleArm() {
  systemArmed = true;
  server.send(200, "text/plain", "System armed");
}

void handleDisarm() {
  systemArmed = false;
  alertActive = false;
  noTone(BUZZER_PIN);
  server.send(200, "text/plain", "System disarmed");
}

void handleReset() {
  armedFullHits = 0;
  disarmedFullHits = 0;
  alertActive = false;
  fullBarLatched = false;
  noTone(BUZZER_PIN);
  server.send(200, "text/plain", "Alert reset");
}

void handleSetLimit() {
  if (server.hasArg("value")) {
    int value = server.arg("value").toInt();
    if (value < 1) value = 1;
    if (value > 20) value = 20;
    triggerLimit = value;
    server.send(200, "text/plain", "Trigger limit updated");
  } else {
    server.send(400, "text/plain", "Missing value");
  }
}

void handleSetMode() {
```

```
  if (server.hasArg("value")) {
    String value = server.arg("value");
    value.toLowerCase();

    if (value == "silent" || value == "regular") {
      alarmMode = value;
      server.send(200, "text/plain", "Alarm mode updated");
    } else {
      server.send(400, "text/plain", "Invalid mode");
    }
  } else {
    server.send(400, "text/plain", "Missing value");
  }
}

// ---------------- Setup ----------------
void setup() {
  Serial.begin(115200);
  delay(1000);

  pinMode(TRIG_PIN, OUTPUT);
  pinMode(ECHO_PIN, INPUT);
  pinMode(BUZZER_PIN, OUTPUT);
  noTone(BUZZER_PIN);

  for (int i = 0; i < NUM_LEDS; i++) {
    pinMode(ledPins[i], OUTPUT);
    digitalWrite(ledPins[i], HIGH);   // active-low OFF
  }

  Wire.begin(21, 22);
  lcd.init();
  lcd.backlight();
  lcd.setCursor(0, 0);
  lcd.print("ESP32 Monitor");
  lcd.setCursor(0, 1);
  lcd.print("Starting...");

  // Wi-Fi
  Serial.println("Connecting to WiFi...");
  WiFi.mode(WIFI_STA);
  WiFi.begin(ssid, password);

  int attempts = 0;
  while (WiFi.status() != WL_CONNECTED && attempts < 30) {
```

```
    delay(500);
    Serial.print(".");
    attempts++;
  }

  Serial.println();

  if (WiFi.status() == WL_CONNECTED) {
    Serial.println("Connected to WiFi!");
    Serial.print("ESP32 IP Address: ");
    Serial.println(WiFi.localIP());

    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("WiFi Connected");
    lcd.setCursor(0, 1);
    lcd.print(WiFi.localIP());
    delay(2000);
  } else {
    Serial.println("Failed to connect to WiFi.");

    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("WiFi Failed");
    lcd.setCursor(0, 1);
    lcd.print("Check Network");
    delay(2000);
  }

  // Web routes
  server.on("/", handleRoot);
  server.on("/data", handleData);
  server.on("/arm", handleArm);
  server.on("/disarm", handleDisarm);
  server.on("/reset", handleReset);
  server.on("/setLimit", handleSetLimit);
  server.on("/setMode", handleSetMode);
  server.begin();

  Serial.println("Web server started.");

  // BLE
  setupBLE();
}
```

```
// ---------------- Main Loop ----------------
void loop() {
  server.handleClient();

  float distance = getSmoothDistance();

  if (distance < 0) {
    currentDistance = 0;
    motionStatus = "No Valid Reading";

    unsigned long now = millis();
    if (displayedLedsOn > 0 && now - lastLedStepTime >= ledStepDelay) {
      displayedLedsOn--;
      lastLedStepTime = now;
    }

    currentLedsOn = displayedLedsOn;
    updateBarGraph(currentLedsOn);
    evaluateTriggerLogic();
    updateLCD();
    handleAlarmSound();

    Serial.println("No valid reading");
    delay(30);
    return;
  }

  currentDistance = distance;

  float clampedDistance = distance;
  if (clampedDistance < MIN_DIST) clampedDistance = MIN_DIST;
  if (clampedDistance > MAX_DIST) clampedDistance = MAX_DIST;

  filteredDistance = (distanceAlpha * clampedDistance) + ((1.0 -
distanceAlpha) * filteredDistance);

  int targetLedsOn = map((int)filteredDistance, MAX_DIST, MIN_DIST, 0,
NUM_LEDS);
  targetLedsOn = constrain(targetLedsOn, 0, NUM_LEDS);

  unsigned long now = millis();
  if (now - lastLedStepTime >= ledStepDelay) {
    if (displayedLedsOn < targetLedsOn) {
      displayedLedsOn++;
      lastLedStepTime = now;
```

```
    } else if (displayedLedsOn > targetLedsOn) {
      displayedLedsOn--;
      lastLedStepTime = now;
    }
  }

  currentLedsOn = displayedLedsOn;
  updateBarGraph(currentLedsOn);

  if (currentLedsOn == NUM_LEDS) {
    motionStatus = "Motion Detected";
  } else {
    motionStatus = "No Motion";
  }

  evaluateTriggerLogic();
  updateLCD();
  handleAlarmSound();

  Serial.printf("D:%.1f B:%d Armed:%d AH:%d DH:%d L:%d Alert:%d
Mode:%s\n",
                currentDistance,
                currentLedsOn,
                systemArmed,
                armedFullHits,
                disarmedFullHits,
                triggerLimit,
                alertActive,
                alarmMode.c_str());

  delay(30);
}
```

HTML Code:

<!DOCTYPE html>

<html>

<head>

<meta charset="UTF-8">

<title>ESP32 Security Monitor</title>

```
<meta name="viewport" content="width=device-width, initial-scale=1">
<style>
 body {
  font-family: Arial, sans-serif;
  text-align: center;
  background: #111;
  color: white;
  margin-top: 40px;
 }

 .card {
  width: 400px;
  max-width: 92%;
  margin: auto;
  padding: 25px;
  border-radius: 16px;
  background: #1e1e1e;
  box-shadow: 0 0 15px rgba(255,255,255,0.1);
 }

 h1 {
  margin-bottom: 20px;
 }

 .status {
  font-size: 28px;
```

```
  font-weight: bold;
  margin: 20px 0;
}

.motion {
  color: red;
}

.nomotion {
  color: limegreen;
}

.armed {
  color: orange;
  font-weight: bold;
}

.disarmed {
  color: deepskyblue;
  font-weight: bold;
}

.alert-on {
  color: red;
  font-weight: bold;
}
```

```
.alert-off {
  color: limegreen;
  font-weight: bold;
}

.reading {
  font-size: 20px;
  margin: 10px 0;
}

.controls {
  margin-top: 20px;
  text-align: left;
}

.controls label {
  display: block;
  margin-top: 15px;
  margin-bottom: 5px;
  font-size: 16px;
}

input, select, button {
  width: 100%;
  padding: 10px;
```

```
  margin-top: 5px;
  border: none;
  border-radius: 8px;
  font-size: 16px;
  box-sizing: border-box;
}

button {
  cursor: pointer;
  background: #333;
  color: white;
  margin-top: 10px;
}

button:hover {
  background: #555;
}

.button-row {
  display: flex;
  gap: 10px;
  margin-top: 10px;
}

.button-row button {
  flex: 1;
```

```
  }
 </style>
</head>
<body>
 <div class="card">
  <h1>ESP32 Security Monitor</h1>

  <div id="status" class="status">Loading...</div>
  <div id="distance" class="reading">Distance: -- cm</div>
  <div id="leds" class="reading">LEDs ON: -- / 10</div>
  <div id="armState" class="reading">System: --</div>
  <div id="alarmMode" class="reading">Alarm Mode: --</div>
  <div id="armedHits" class="reading">Armed Hits: --</div>
  <div id="disarmedHits" class="reading">Disarmed Hits: --</div>
  <div id="triggerLimit" class="reading">Trigger Limit: --</div>
  <div id="alertState" class="reading">Alert: --</div>

  <div class="controls">
   <label for="limitInput">Set Trigger Limit</label>
   <input type="number" id="limitInput" min="1" max="20" value="3">

   <button onclick="setLimit()">Set Limit</button>

   <label for="modeSelect">Alarm Mode</label>
   <select id="modeSelect" onchange="setMode()">
    <option value="silent">Silent Alarm</option>
```

```
    <option value="regular">Regular Alarm</option>
   </select>

   <div class="button-row">
    <button onclick="sendCommand('arm')">Arm</button>
    <button onclick="sendCommand('disarm')">Disarm</button>
   </div>

   <div class="button-row">
    <button onclick="sendCommand('reset')">Reset Alert</button>
    <button onclick="updateData()">Refresh</button>
   </div>
  </div>
</div>

<script>
 function updateData() {
  fetch('/data')
   .then(response => response.json())
   .then(data => {
    const statusEl = document.getElementById('status');
    statusEl.textContent = data.status;

    if (data.status === "Motion Detected" || data.alertActive === true) {
     statusEl.className = "status motion";
    } else {
```

```
  statusEl.className = "status nomotion";
}

document.getElementById('distance').textContent =
  "Distance: " + data.distance + " cm";

document.getElementById('leds').textContent =
  "LEDs ON: " + data.ledsOn + " / 10";

const armStateEl = document.getElementById('armState');
armStateEl.textContent = "System: " + (data.armed ? "ARMED" : "DISARMED");
armStateEl.className = "reading " + (data.armed ? "armed" : "disarmed");

document.getElementById('alarmMode').textContent =
  "Alarm Mode: " + data.alarmMode;

document.getElementById('armedHits').textContent =
  "Armed Hits: " + data.armedHits;

document.getElementById('disarmedHits').textContent =
  "Disarmed Hits: " + data.disarmedHits;

document.getElementById('triggerLimit').textContent =
  "Trigger Limit: " + data.triggerLimit;

const alertStateEl = document.getElementById('alertState');
```

```
    alertStateEl.textContent = "Alert: " + (data.alertActive ? "ACTIVE" : "CLEAR");
    alertStateEl.className = "reading " + (data.alertActive ? "alert-on" : "alert-off");

    document.getElementById('limitInput').value = data.triggerLimit;
    document.getElementById('modeSelect').value = data.alarmMode;
  })
  .catch(error => {
    console.log("Error:", error);
  });
}

function sendCommand(cmd) {
 fetch('/' + cmd)
  .then(response => response.text())
  .then(result => {
    console.log(result);
    updateData();
  })
  .catch(error => {
    console.log("Error:", error);
  });
}

function setLimit() {
 const value = document.getElementById('limitInput').value;
 fetch('/setLimit?value=' + value)
```

```
    .then(response => response.text())
    .then(result => {
      console.log(result);
      updateData();
    })
    .catch(error => {
      console.log("Error:", error);
    });
}

function setMode() {
  const mode = document.getElementById('modeSelect').value;
  fetch('/setMode?value=' + mode)
    .then(response => response.text())
    .then(result => {
      console.log(result);
      updateData();
    })
    .catch(error => {
      console.log("Error:", error);
    });
}

updateData();
setInterval(updateData, 1000);
</script>
```

</body>

</html>

Commands for BLE:

ARM

DISARM

STATUS

RESET

SILENT

REGULAR

Serial Monitor: